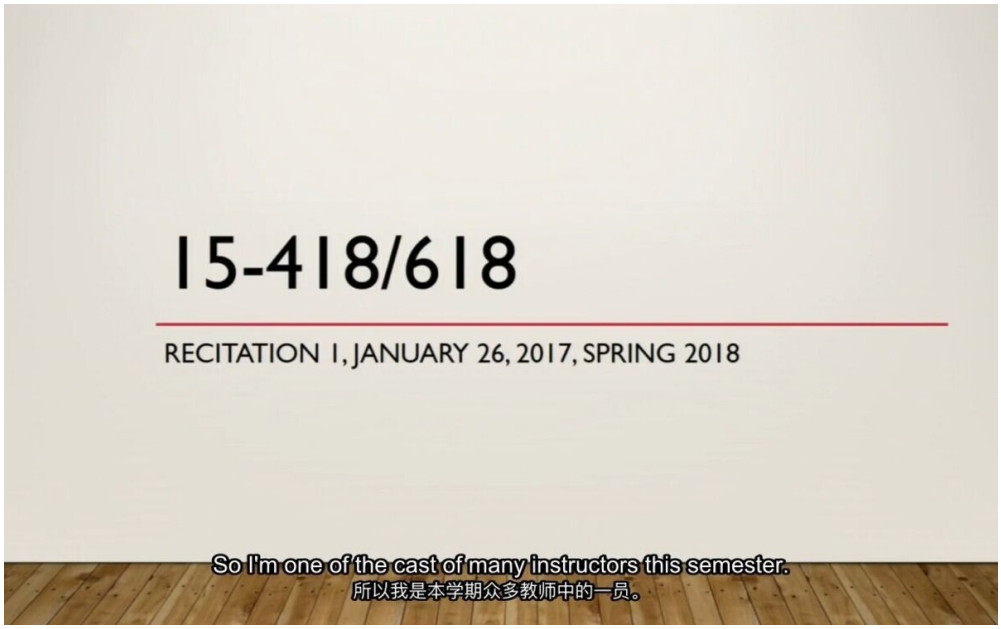
课程笔记

性能调优的实用工具：从 /proc 到向量化

CMU 15-418/618 并行计算机架构与编程 · Recitation（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（Greg Kesden 讲授，B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 2 分 25 秒
视频链接：`https://www.bilibili.com/video/BV1uh4y1Q73E/`

目录

1 先看清机器：/proc 这一手资料 **2**
1.1 /proc 是什么 . . . 2
1.2 进程目录与 cpuinfo . . . 2
1.3 几个必须读对字段 . . . 4
1.4 还能快速查什么 . . . 5
1.5 本章小结 . . . 5

2 读架构手册：功能单元与流水线 **5**
2.1 细节都在手册里 . . . 5
2.2 流水线的心智模型 . . . 6
2.3 延迟、发射时间与容量 . . . 6
2.4 本章小结 . . . 7

3 优化的起点：找到内层循环 **8**
3.1 一个贯穿全课的代码例子 . . . 8
3.2 Amdahl 的饼图视角 . . . 8
3.3 内层循环为什么是首要目标 . . . 9
3.4 先测基线，不要凭直觉优化 . . . 10
3.5 本章小结 . . . 11

4 手工改造内层循环 **11**
4.1 从最贵的操作入手 . . . 11
4.2 循环展开 . . . 12
4.3 本章小结 . . . 12

5 读汇编：确认编译器做了什么 **13**
5.1 从汇编里看真实指令 . . . 13
5.2 三个人、三个加速比 . . . 13
5.3 本章小结 . . . 13

6 向量化与基准测量的纪律 **14**
6.1 向量化带来数量级差异 . . . 14
6.2 阻碍向量化的常见因素 . . . 14
6.3 本章小结 . . . 15

7 基准测量的纪律 **15**
7.1 测量本身要可靠 . . . 15
7.2 本章小结 . . . 16

8 总结与延伸 **16**

1 先看清机器：/proc 这一手资料

这堂习题课由 Greg Kesden 讲授，内容是把前几讲的方法落到实际工具上。开场他就说明这是一堆与课程相关的杂项，主题围绕性能优化。

1.1 /proc 是什么

讲者先问有多少人看过 `/proc/cpuinfo`。他从历史讲起：过去想从内核取信息，必须做系统调用，而系统调用本身有固定开销。

现代操作系统提供 `/proc`，它是一个虚拟文件系统：不是真实文件系统，但看起来像。内核把大量状态导出，伪装成文件系统的形式，因为这种形式对人方便，对程序也方便——打开、读取、解析都容易。这样做同时省掉了每次都做系统调用的开销。

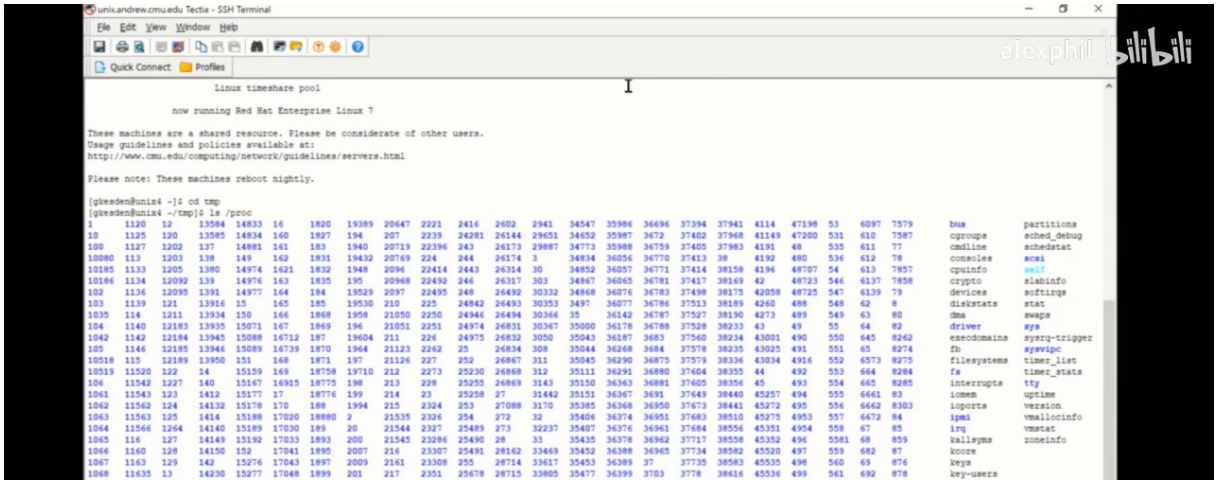


图 1: 在终端中浏览 /proc：内核状态以文件形式呈现[1]

1.2 进程目录与 cpuinfo

`/proc` 下一串以数字命名的目录代表进程，数字就是进程 ID。对某个编号做 `ls`，就能看到那个进程的各项信息。讲者现场演示时那个进程已经退出，于是换了一个。

真正与性能相关的是 `/proc/cpuinfo`。它暴露处理器数量、型号与各种属性。

[1] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

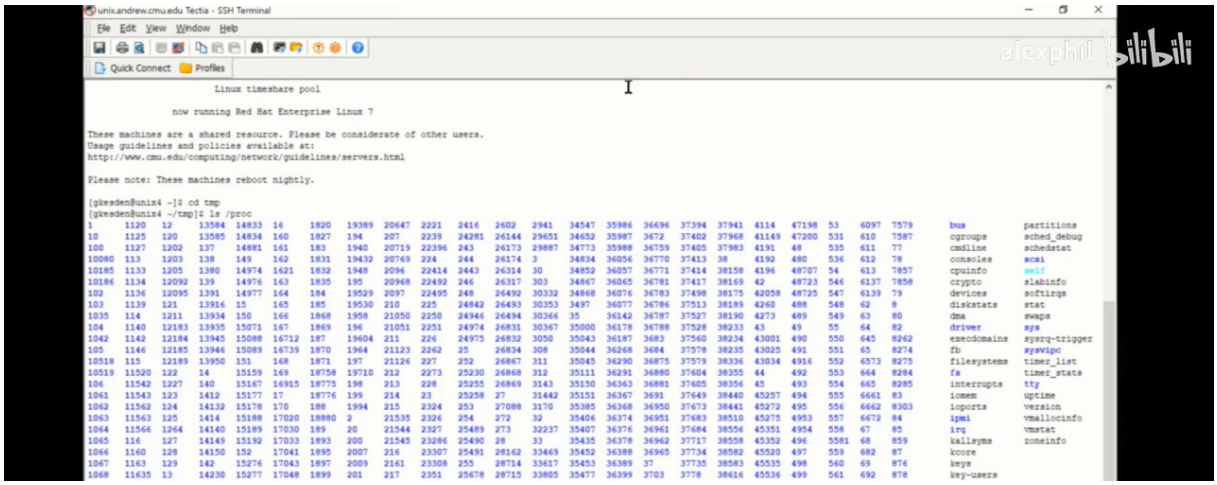


图 2: 列出 /proc 下以进程号命名的目录[2]

按逻辑处理器数规划并行度会高估算力

超线程共享同一核心的执行单元与缓存，两条逻辑线程的合计吞吐明显低于两个独立物理核。把 40 个逻辑处理器当作 40 份独立算力来切分工作，会得到过度乐观的预期，实测加速比通常远低于设想的倍数。

讲者现场展示了一台机器：报告 40 个处理器，而 siblings 是 20。他让学生猜这意味着什么。答案是有大量超线程——每核两条线程，于是 20 个物理核被操作系统看成 40 个。

处理器数与 siblings 数不一致就说明有超线程

cpuinfo 中 `processor` 的条目数是操作系统看到的逻辑处理器数，`siblings` 是同一物理封装内的逻辑处理器数。两者不等时，你拿到的核数要按物理核重新算。把超线程当成独立核心来规划并行度，会高估可用算力。

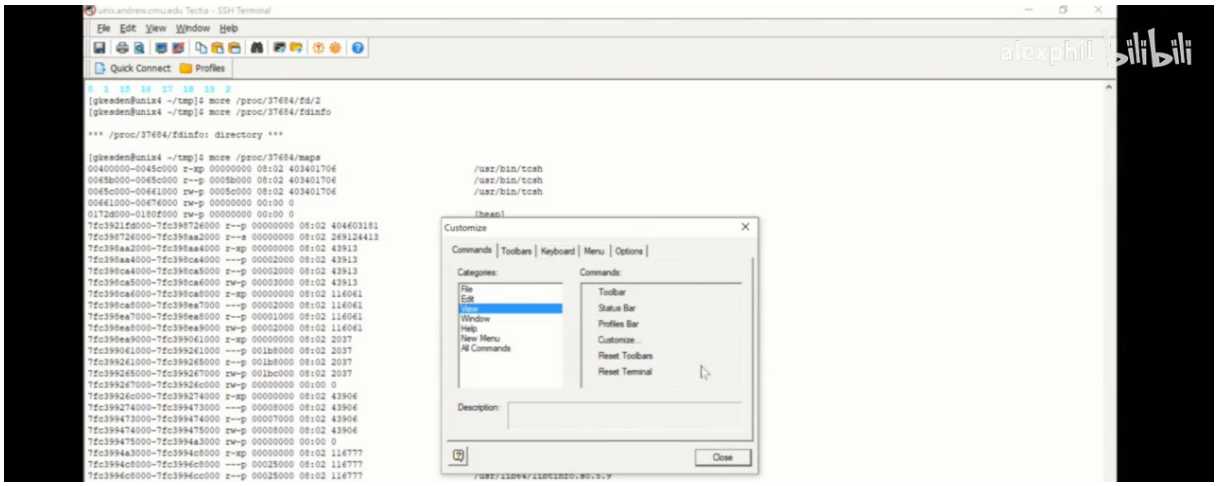


图 3: 查看 cpuinfo 的终端会话[3]

[2]视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

1.3 几个必须读对字段

讲者逐项解释容易误读的字段。

model name 通常标注处理器的最高频率。而 cpu MHz 是当前频率。现代处理器有降频省电机制，也有短时间的睿频模式，因此这两个数字常常不一致。看到 cpu MHz 偏低不代表处理器变慢了。

cache size 只显示最外层缓存，通常是所有核心共享的 L3。各核私有的 L1、L2 不在这一项里显示。

physical id 是插槽编号。多处理器系统里两个处理器各带若干核心时，这个字段能区分它们。

flags 列出处理器的能力，例如这门课一直在用的 AVX2 指令。

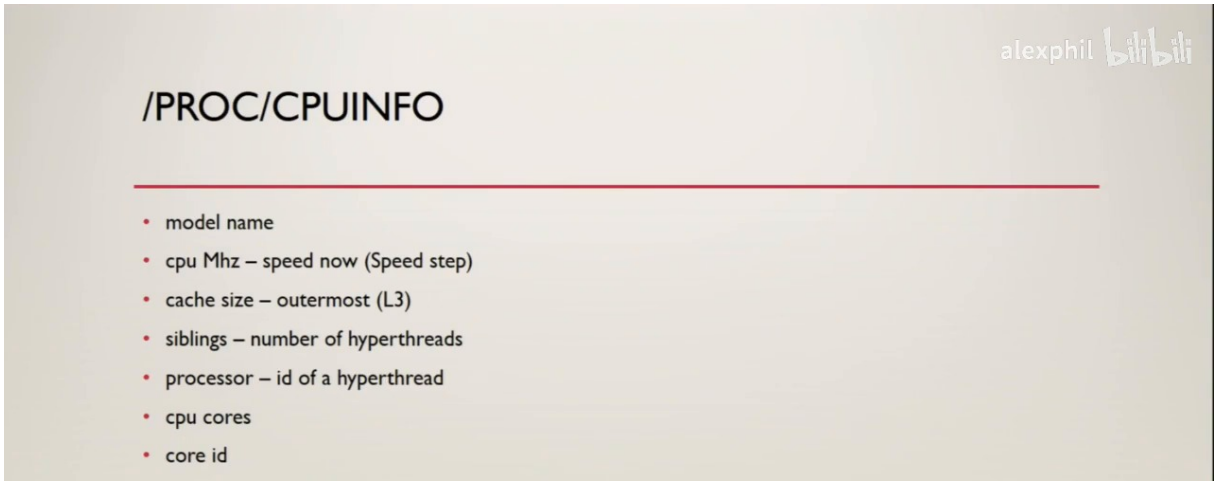


图 4: cpuinfo 关键字段：型号、当前频率、最外层缓存、siblings[4]



图 5: cpuinfo 中的 physical id 与 flags[5]

[3] 视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

[4] 视频画面时间区间：00:11:40–00:11:55。若为裁剪图，时间区间仍对应原视频画面。

[5] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

信息多不等于有用

讲者特意提醒：这里是一座宝库，但打开后很容易被海量字段带跑。做性能工作时应该带着具体问题来查——我要知道核数、当前频率、是否有 AVX2——而不是通读一遍。

1.4 还能快速查什么

讲者给出了一份”快速查”清单，其中包括内存带宽与功耗这类直接影响并行上限的量。他展示的一台机器标称约 76.8 GB/s 带宽、140 瓦功耗。

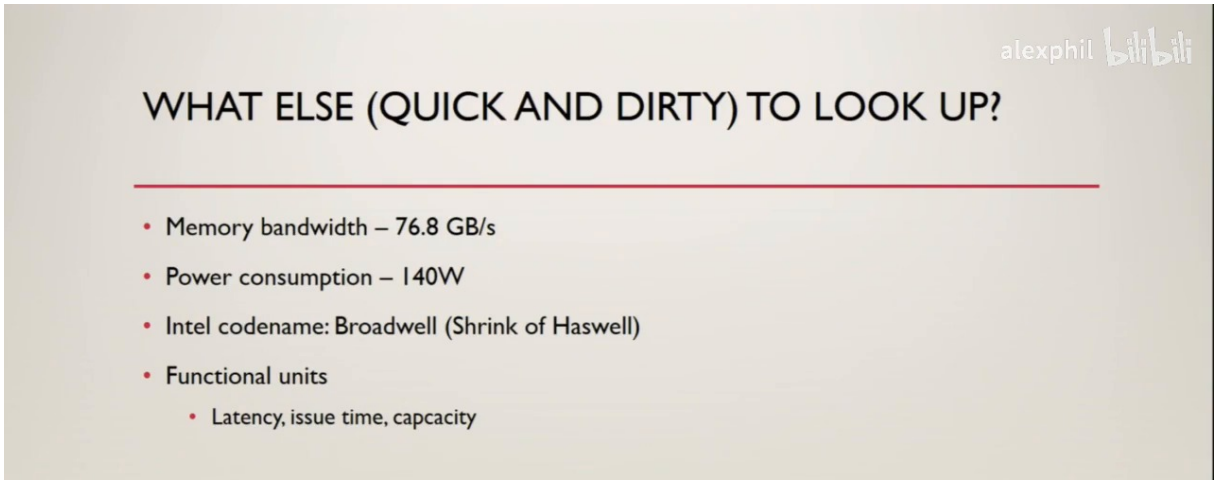


图 6: 还能快速查到什么：内存带宽与功耗等参数[6]

1.5 本章小结

`/proc` 把内核信息做成文件，避免了反复系统调用的开销。读取时要抓住几个真正影响并行决策的字段：逻辑处理器数、siblings、当前频率、最外层缓存、插槽编号、指令集 flags。其中处理器数与 siblings 不一致，是判断超线程最直接的线索。

知道了机器长什么样，下一步是知道它为什么这样设计。讲者翻开了处理器架构手册。

2 读架构手册：功能单元与流水线

2.1 细节都在手册里

讲者推荐了两份资料：Agner Fog 的微架构文档，以及 2-13 教材第五章。他把这称为”细节里的魔鬼”——这些文档里有关功能单元、延迟、端口的具体数字。

[6] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。



图 7: 细节藏在架构手册里：推荐的参考资料[7]

2.2 流水线的心智模型

讲者重申了这门课使用的处理器模型：像一条装配线。指令进来、被译码、被派发，在没有数据依赖或其他结构性冒险时可以乱序执行，最后进入提交单元，从那里按序把结果落实到真实状态。

还有一个 2-13 课程里没强调的事实：寄存器在处理器内部有很多份副本，正是这些副本让并行得以发生。开启超线程后，处理器不只是并行执行一条指令流，它能同时跟踪多条指令流。

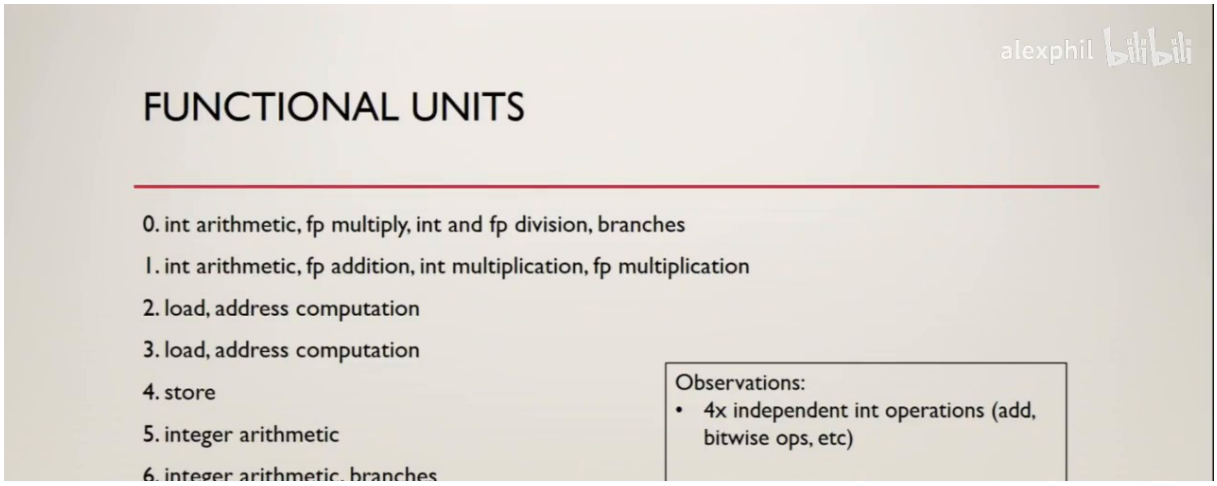


图 8: 处理器的功能单元及其分工[8]

2.3 延迟、发射时间与容量

讲者用一张表把三个常被混淆的量分开。

延迟是这条指令从开始到结果可用所需的时间。发射时间是相邻两条同类指令之间至少要隔多久，也就是吞吐的倒数。容量是有多少个功能单元能支持这类操作。

[7] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

他指出了一个过去的简化不再成立的地方：早年间说”能做四个加法”就足够描述，而现在部分功能单元承担一个操作篮子，能做一个或另一个但不能同时做。因此简单地相加容量会高估实际能力。

若只考虑单一功能单元，稳定状态下的吞吐由发射时间决定：

$$\text{Throughput} = \frac{\text{单元数量}}{\text{发射时间}}$$

- 单元数量：能执行该操作的功能单元个数；
- 发射时间：相邻两条同类指令的最小间隔，单位为周期；
- 结果为单位周期内可完成的该操作次数。

注意分母是发射时间而不是延迟：延迟决定单条依赖链的长度，发射时间才决定吞吐。

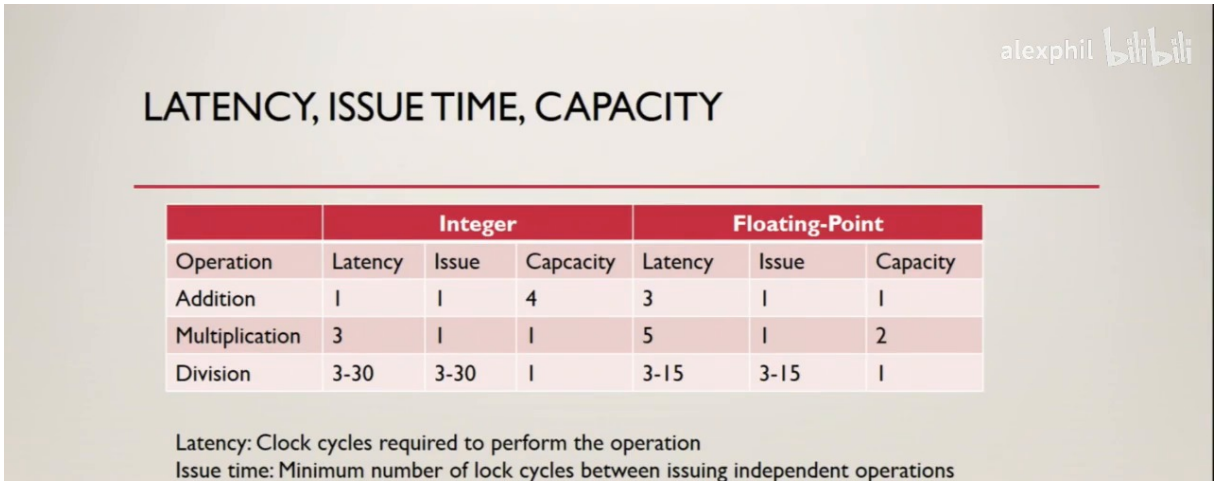


图 9: 延迟、发射时间与容量：三个常被混淆的量[9]

三个量的用途各不相同

延迟决定依赖链的长度，影响单条关键路径的耗时；发射时间决定单位时间能塞进多少条同类指令，影响吞吐；容量决定同类型操作的并行上限。优化时先判断瓶颈属于哪一种，三者的改进手段完全不同。

2.4 本章小结

架构手册给出功能单元的真实参数，流水线模型解释了乱序执行与按序提交。延迟、发射时间、容量是三个必须区分的量；现代处理器中功能单元能力重叠，不能简单相加估计吞吐。

有了机器的参数与模型，就可以回到程序本身。下一章用一段具体代码演示优化从哪一步开始。

[9] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

3 优化的起点：找到内层循环

3.1 一个贯穿全课的代码例子

讲者用的是计算 sin 的泰勒级数近似。这段代码在前几讲出现过，因为它简单、能放在一页幻灯片上。

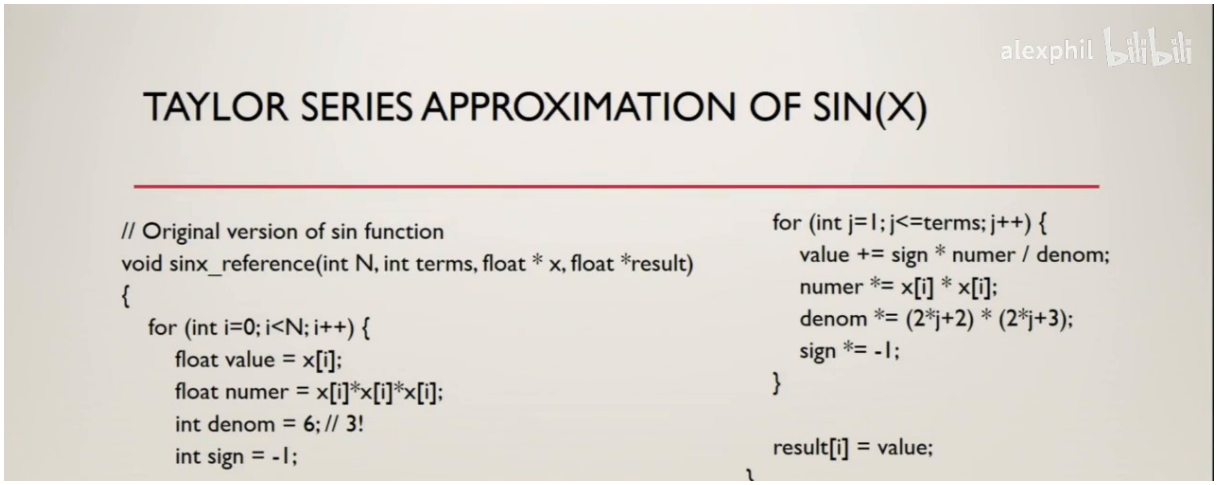


图 10: sin(x) 泰勒级数近似的完整代码[10]

3.2 Amdahl 的饼图视角

讲者问学生 Amdahl 定律是什么，然后给出他更喜欢的表述方式：把程序的活动想成一张饼图，你在优化其中某些切片。如果挑中的切片很小，你能得到的改进也只能很小。

若某个活动只占全部时间的一成，你把它优化到零，也只省下一成时间。由此得到一条排序原则：在常见活动上的小优化，收益往往大于在罕见活动上的大优化。写成式子：

$$S_{\text{total}} = \frac{1}{(1-f) + \dfrac{f}{k}}$$

- S_{total}：整段程序获得的加速比；
- f：被优化部分占总时间的比例；
- k：该部分自身被加速的倍数。

当 $f = 0.9$ 而 $k \to \infty$ 时，S_{total} 的上界只有 10；而当 $f = 0.005$ 时，无论 k 多大，整体几乎不变。

Amdahl 定律的通用形式

这条定律原本用于并行化的加速上界，但它对纯串行程序的优化同样成立：某段代码占运行时间比例 f，则它对总体加速的贡献不超过 $1/((1-f)+f/k)$，其中 k 是该段被加速的倍数。因此优化的顺序应当按占比从大到小排，而不是按改起来是否顺手排。

[10] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

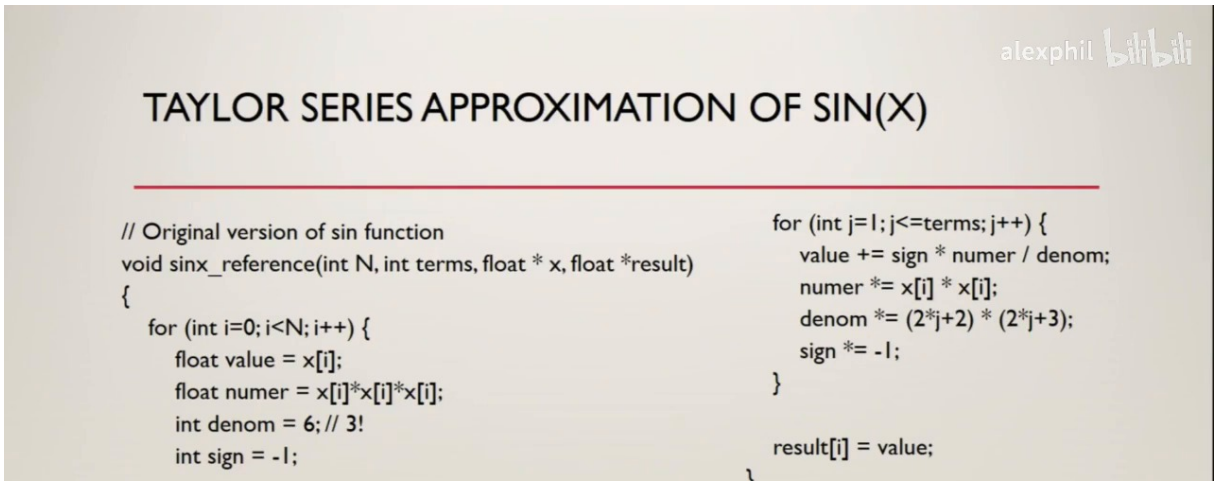


图 11: 定位内层循环：真正被反复执行的部分[11]

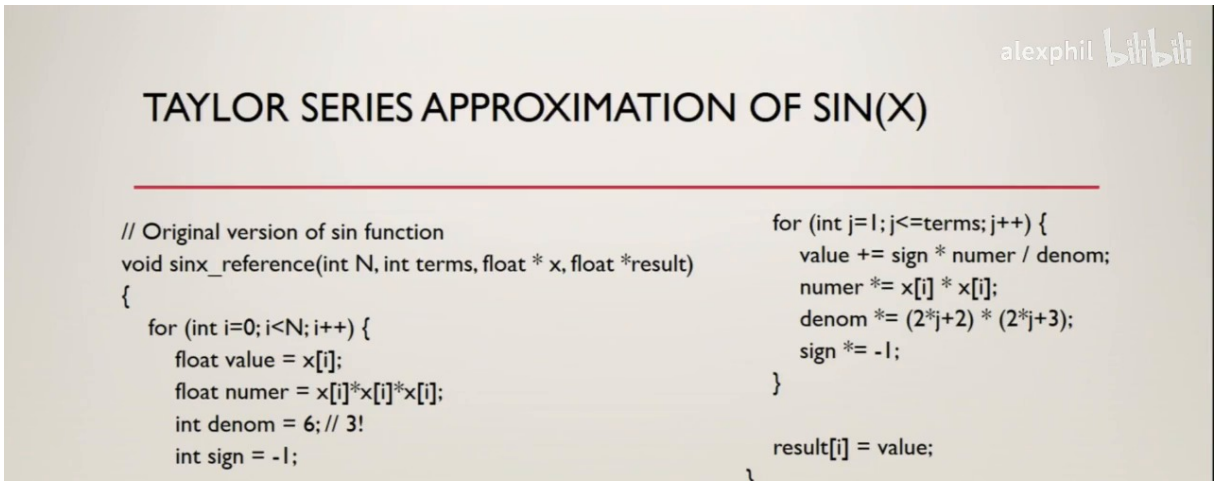


图 12: Amdahl 的饼图视角：优化的收益上限由占比决定[12]

3.3 内层循环为什么是首要目标

这段代码的循环是嵌套的：外层执行 n 次，内层执行 terms 次。因此内层循环体被执行 $n \times \text{terms}$ 次，这就是所谓的内层工作循环。

讲者的对比很有力：如果把开头的三条指令彻底消掉，整个程序只快了 0.5%，因为那三条不常执行。反过来，在内层循环里省一件事，这个节省会被执行次数放大。

内层循环体的执行次数是两层循环次数之积：

$$C_{\text{inner}} = n \times \text{terms}$$

- C_{inner}：内层循环体被执行的次数；
- n：外层循环次数，即输出元素个数；
- terms：泰勒级数保留的项数。

[11] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。
[12] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

这解释了为什么同样省一条指令，放在内层循环与放在外层开头的效果相差几个数量级。

3.4 先测基线，不要凭直觉优化

讲者引用了上一讲 Todd 的说法：拿到初始测量非常重要。很多人会在脑子推演” 这里贵、那里也贵”，然后还没把代码写清楚就开始优化。他明确说不要这样。

他们的基线是每元素约 7 纳秒。这个数字本身没有绝对意义，它是后面每次改动的参照点。

每元素成本由总时间除以元素数得到：

$$T_{\text{elem}} = \frac{T_{\text{total}}}{N}$$

- T_{elem}：分摊到每个元素的时间，本例约 7 纳秒；
- T_{total}：一次完整运行的总耗时；
- N：处理的元素总数。

用每元素成本而不是总时间做比较，可以在元素数变化时仍保持可比。

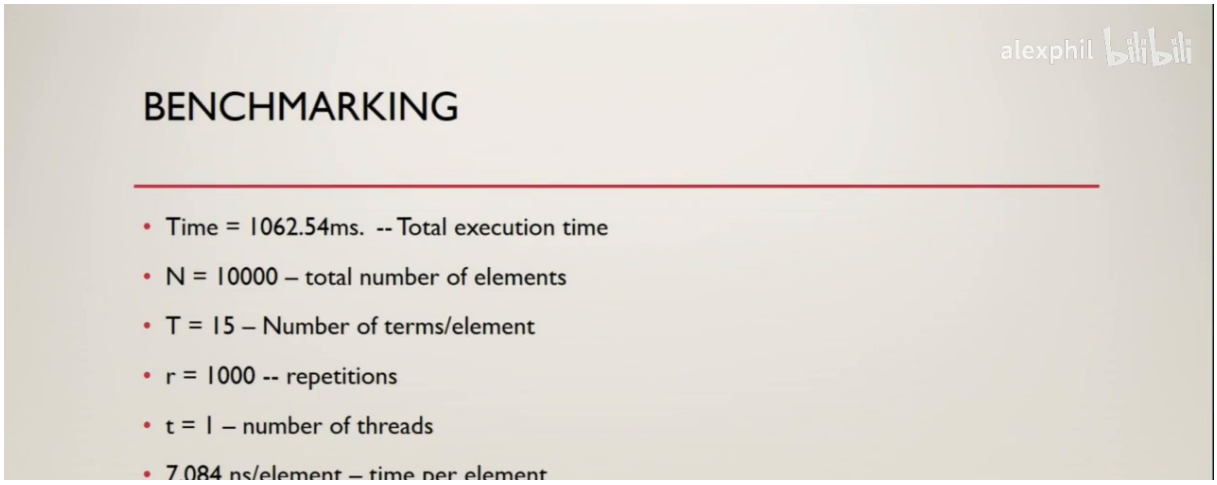


图 13: 基准测量：先拿到初始数字再优化[13]

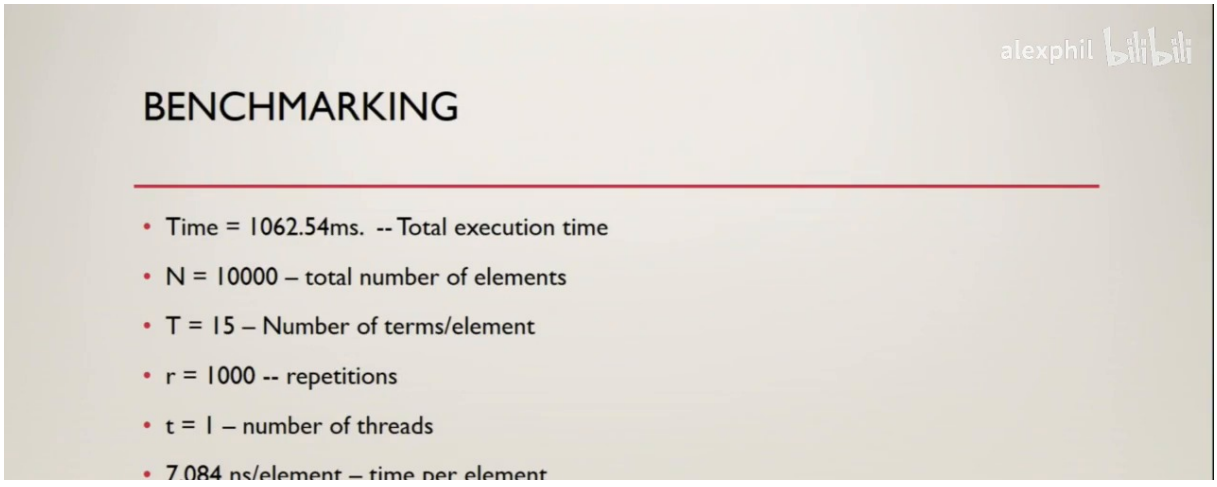


图 14: 基准测量的细节：总时间、元素数与每元素成本[14]

[13]视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。

没有基线就无法判断改动是否有效

一次改动带来百分之几的差异，可能与测量噪声同量级。没有稳定的基线，你无法区分真实改进与随机波动，也无法在多次迭代中追踪哪一步真正起了作用。基线必须在优化开始之前建立，并且用同样的方式重复测量。

3.5 本章小结

优化从内层工作循环开始，因为它的执行次数最多，任何节省都会被放大。判断依据是 Amdahl 的饼图视角：占比决定收益上限。动手前必须先建立基线（本例为每元素约 7 纳秒），否则无法判断改动是否真的有效。

定位了目标之后，接下来看两个方向：人工改写代码，以及依赖编译器。

4 手工改造内层循环

4.1 从最贵的操作入手

讲者审视内层循环，找出最贵的部分：乘法与除法，其中除法更贵。因此初期目标是减少这两类操作的次数。

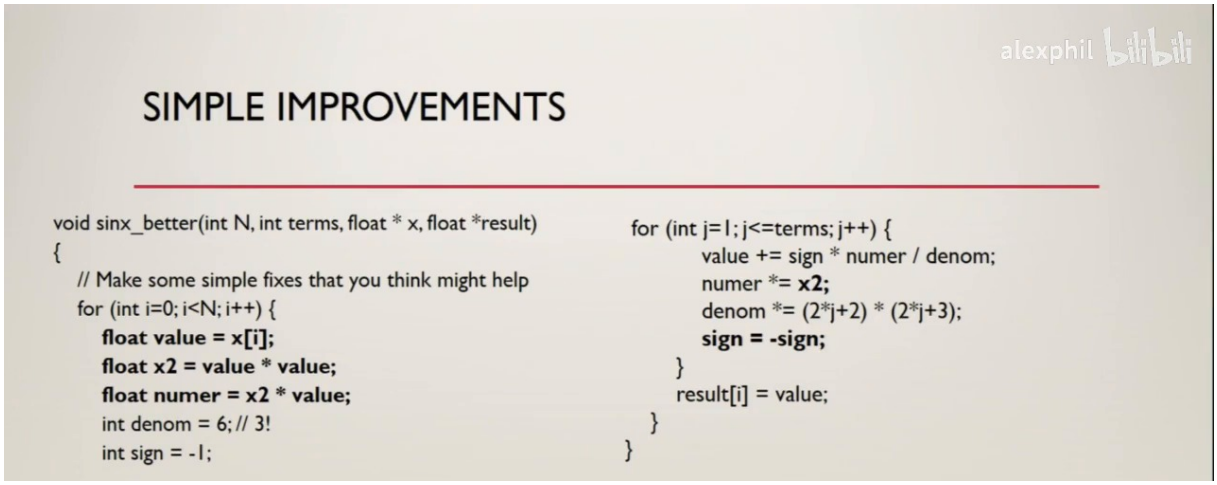


图 15：简单改进：预先计算阶乘倒数，把除法转为乘法[15]

预先计算把循环内的工作搬到循环外

在循环外预先算好阶乘倒数，等于把每轮都要重复的计算变成一次性的表。这类改写的适用范围很广：任何只依赖循环不变量的计算都可以移出去。代价是多一份存储，收益是内层循环少一整类昂贵操作。

最直接的一步是预先算好各阶乘的倒数，避免在循环里反复做除法。这类改写不需要改变算法结构，属于低风险高回报的改动。

[14] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:38:30–00:38:45。若为裁剪图，时间区间仍对应原视频画面。

讲者还提到另一类改写的效果量级：把表达式重组之后，某些情况下能拿到 180% 的改善，而另一些只带来 2% 的微小增益。这个对比说明改写的收益高度依赖它在内层循环里的位置与编译器能否配合。

4.2 循环展开

下一步是循环展开：减少循环条件判断的次数，增加每次迭代内的工作量，从而避开分支代价。

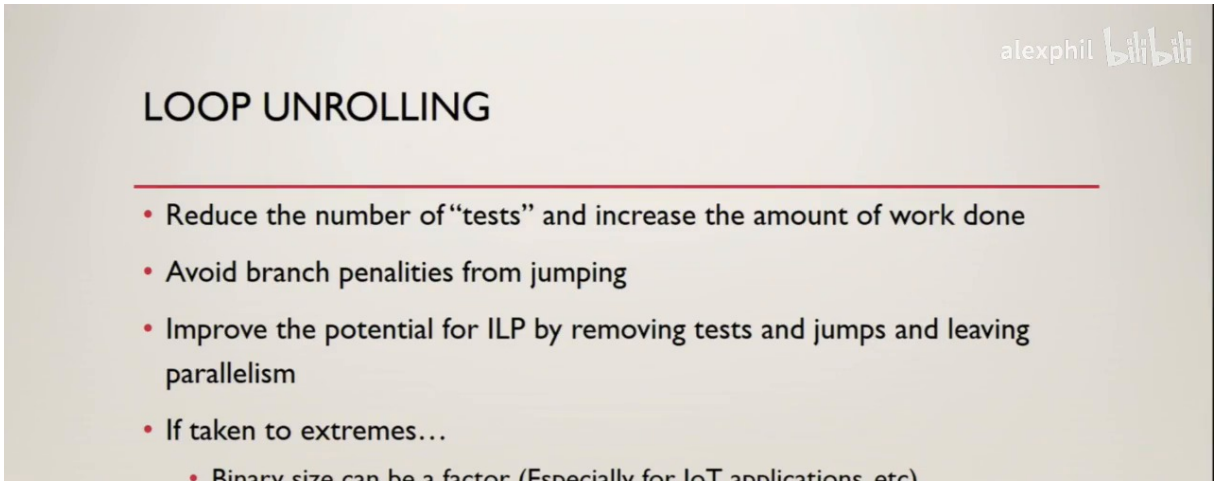


图 16: 循环展开：减少判断次数并增加单次迭代的工作量[16]

讲者也指出了代价：展开后需要更多变量来保存中间结果，寄存器压力上升。寄存器不够用时就会溢出到内存，反而变慢。

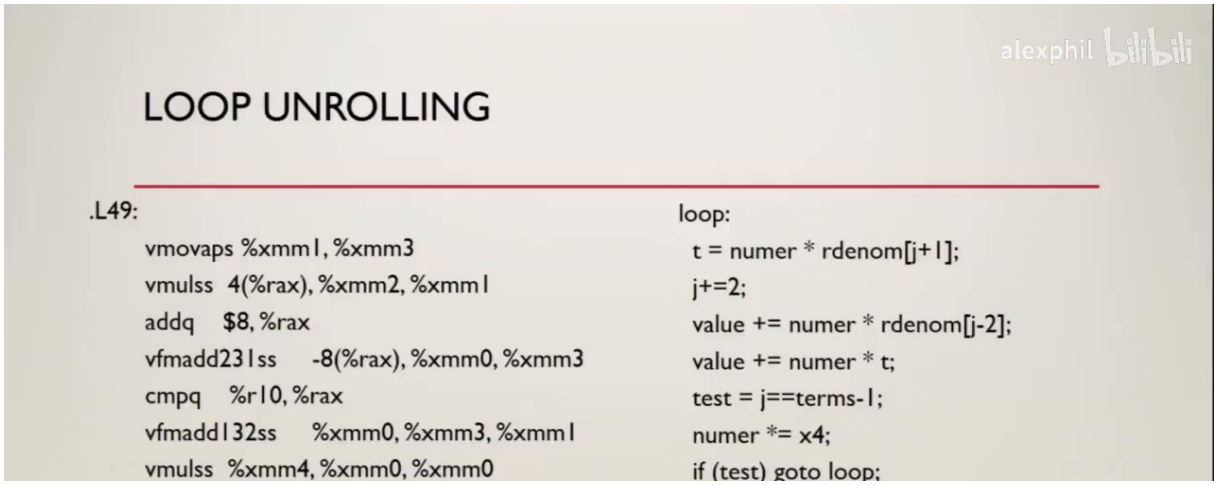


图 17: 展开后的汇编：更多寄存器被占用[17]

4.3 本章小结

手工改造内层循环的两个方向是减少昂贵操作与循环展开。前者把除法转成乘法，后者减少分支判断，但展开会增加寄存器压力，溢出到内存反而更慢。这些改动是否真的生效，需要另一种手

[16] 视频画面时间区间：00:41:00–00:41:15。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:46:00–00:46:15。若为裁剪图，时间区间仍对应原视频画面。

段来确认。

源码看不出编译器做了什么，因此需要直接阅读机器生成的指令。

5 读汇编：确认编译器做了什么

5.1 从汇编里看真实指令

讲者展示了内层循环的汇编，其中出现了 `vfmadd231ss` 这类融合乘加指令。

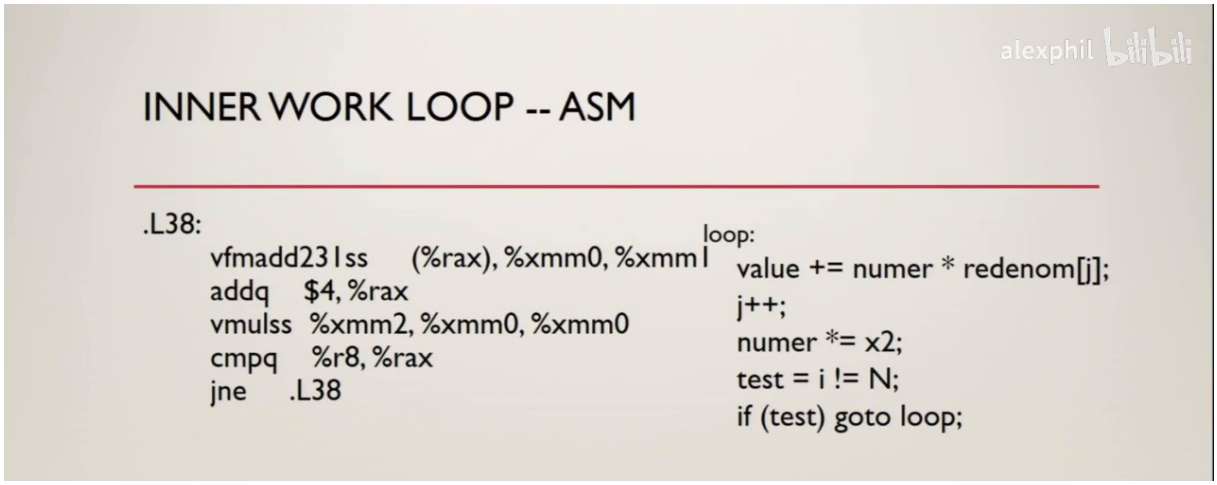


图 18: 内层工作循环的汇编代码：融合乘加等指令[18]

汇编是验证优化是否生效的唯一直接证据

阅读源码无法知道编译器做了什么。要确认循环被展开、乘加被融合、或者代码没有被向量化，只能看生成的汇编。这也是判断”我的改动被编译器接受了”与”编译器另有打算”的分界线。

5.2 三个人、三个加速比

讲者分享了这段代码的实战对照。他自己第一版优化做到 25 倍加速；看到 Randolph 去年这堂课的笔记写着 80 倍；于是他回去重新努力，做到 40 倍；最后还是忍不住看了笔记。

同一个优化目标，不同人的结果可以差数倍

25 倍与 80 倍的差距不来自编译器版本，而来自对瓶颈的理解深度。这说明优化不是套用技巧清单，而是逐步建立对这段代码在硬件上如何执行的准确认识。每一次对照更高水平的结果，都能暴露自己模型里的错误假设。

5.3 本章小结

手工改造的两大方向是减少昂贵操作（除法转乘法）与循环展开（减少判断），后者受寄存器容量限制。看汇编是确认这些改动是否生效的唯一直接方法。同一段代码的优化结果可以差数倍，差别在于对瓶颈的理解。

[18] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

手工手段的收益有上限。真正数量级的提升来自另一种方式：让处理器用一条指令处理多个数据。

6 向量化与基准测量的纪律

6.1 向量化带来数量级差异

讲者给出了 ISPC 版本与未向量化版本的直接对照：ISPC 代码每元素 0.63 纳秒，未向量化版本 7.16 纳秒。

这大约是十倍的差距。而这个版本恰恰是增量最小的方向——讲者说向量化很简单，只换来 5.40 倍加速；而传统的手工优化做得很辛苦，换来的是 15 倍量级。

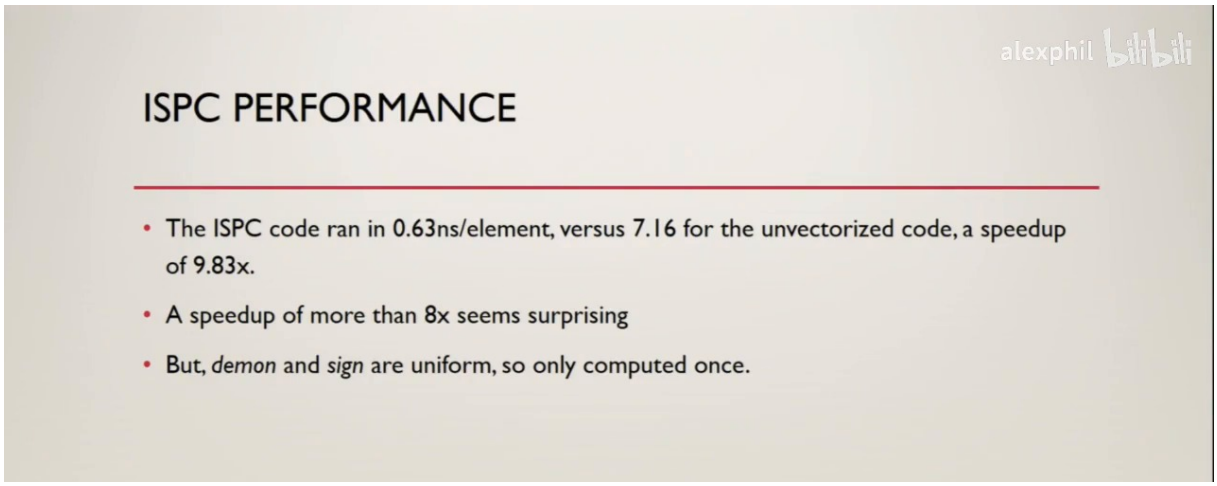


图 19: ISPC 版本与未向量化版本的性能对照[19]

向量化的投入产出比通常优于手工调优

单条指令处理多个数据，等于在不增加指令数的情况下提高吞吐。对结构规整的内层循环，重写成可向量化的形式往往只需少量改动，却能得到数倍提升。相比之下，手工展开、手工调度寄存器需要大量试错，收益还受寄存器数量限制。

6.2 阻碍向量化的常见因素

讲者也提到了让编译器无法向量化的因素：循环体内存在条件分支，或者编译器无法证明两个指针指向不重叠的内存。后者可以用语言特性（如 restrict 限定）来表达。

[19] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

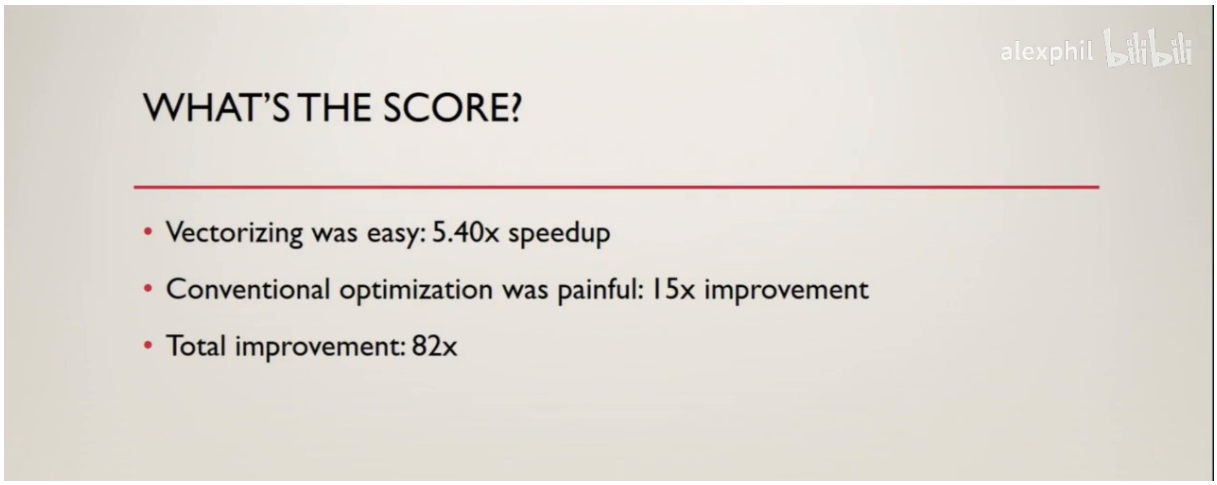


图 20: 用语言特性帮助编译器消除顾虑，从而允许向量化[20]

编译器向量化的三道门槛

第一，迭代必须互相独立，不能有跨迭代依赖；第二，循环体内最好没有数据相关的分支，否则只能靠掩码执行、效率减半；第三，编译器必须能证明写目标与读来源不重叠。三者缺一，向量化就会失败或收益打折。

6.3 本章小结

向量化用一条指令处理多个数据，是投入产出比最高的方向：本例中 ISPC 版本每元素 0.63 纳秒，而未向量化版本为 7.16 纳秒，相差约十倍。它的三道门槛是迭代独立、无数据相关分支、指针不重叠。门槛不满足时编译器会放弃向量化，或者只能靠掩码执行而效率减半。

改动做完之后还要能证明它有效。这就把问题从改写代码转向了测量本身。

7 基准测量的纪律

7.1 测量本身要可靠

讲者最后强调测量本身要可靠。基准程序要避免被优化掉——如果计算结果没人用，编译器可能把整段计算删除，测出来的是零。

时间测量要用足够精度的时钟或性能计数器。多次运行取稳定值，并注意第一次运行的偏差（冷缓存、页错误、频率尚未升上去）。

个位数百分比的改善不可靠

讲者提到某类改动只带来 2% 的改善。在存在频率波动、缓存状态差异、系统噪声的环境里，这个量级的差异完全可能是测量误差。要宣称一次优化有效，改善幅度必须显著大于重复测量之间的波动范围。

[20] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

7.2 本章小结

向量化用一条指令处理多个数据，是投入产出比最高的方向，前提是迭代独立、无分支、指针不重叠。基准测量本身也需要纪律：避免计算被优化掉、用足够精度计时、多次运行、只相信显著大于噪声的改善。

到这里工具与方法都已交代。最后是讲者关于应有的态度的一句提醒。

8 总结与延伸

讲者的收尾很朴素：把风格做到最好，但不要在这上面花三周；然后转向真正的工作。

这一讲没有提出新的并行理论，它提供的是把前面几讲落到机器上的通道。`/proc` 让你知道手上是多少物理核、有没有超线程、频率是多少、支持哪些指令集。架构手册告诉你功能单元的延迟、发射时间与容量，这三个量决定了瓶颈性质。优化从内层循环开始，判据是 Amdahl 的饼图视角——占比决定收益上限。手工手段包括减少除法、循环展开，受寄存器容量约束，效果必须靠看汇编确认。向量化通常性价比最高，但有迭代独立、无分支、指针不重叠三道门槛。而所有这一切的前提是可靠的基准测量。

本讲义在讲者材料之上做的压缩，有三点需要与讲者原话区分：

- 这一讲的工具有明确的移植边界。/proc 是 Linux 专有；在 Windows 或 macOS 上没有对应物，那里要用系统 API 或平台专用工具。它不是通用技能，而是 Linux 环境下的必备技能。
- 加速比的对照（25 倍、40 倍、80 倍）说明的是理解深度而非技巧数量。三人的差异来自对同一段代码瓶颈认识的精确程度，这提示优化工作的重心应当放在建立准确的性能模型上。
- 优化存在应当停手的时刻。讲者明确说不要在代码风格上花三周。当边际收益低于测量噪声、或低于继续投入的机会成本时，继续优化就是浪费。

这堂课的边界在于它是习题课性质，内容按” 学生实际会用到什么” 选取，不追求体系完整。因此本文给出的是工具清单与优先级顺序，不含完整的微架构分析。

两个可以直接从本讲数据提出的下一步问题：其一，向量化带来十倍级提升，而手工优化只有数倍，那么在什么条件下手工优化仍然值得做——是否只在无法向量化的不规则计算上？其二，寄存器压力的临界点在哪里：循环展开几次之后，溢出到内存的代价会吞掉减少分支的收益？这两个问题都需要下一讲要处理的局部性与缓存层次来回答。